P4 Project: Service Function Chaining with NSH, MPLS Tunnels, and SFF Proxies

Programmable Networks

1 Project Overview

The goal of this project is to design and implement, in P4, a simplified Service Function Chaining (SFC) architecture based on the Network Service Header (NSH). The project focuses on the implementation of an SFC classifier and two Service Function Forwarders (SFFs). The SFFs must also act as proxies for Service Functions that are not NSH-aware.

Packets entering the SFC domain are classified according to predefined traffic policies. Matching packets are encapsulated with NSH and transported across the overlay through MPLS tunnels. Service Functions receive only ordinary Ethernet/IPv4/TCP packets. Therefore, before sending a packet to a Service Function, the SFF must remove the SFC encapsulation. When the packet returns from the Service Function, the SFF must restore the appropriate NSH context and continue the service chain.

Return traffic is not processed by the service chains. Packets travelling in the reverse direction must be forwarded directly over IPv4, following the shortest path toward the destination.

2 Reference Topology

The reference topology is shown in Figure 1. Nodes `A`, `D`, and `E` are the main SFC-related nodes. Node `B` acts as the classifier. Nodes `D` and `E` act as SFFs and are connected to the Service Functions. Nodes `h1`, `h2`, `h3`, and `h4` are end hosts.

The following service-chain policies must be implemented:

- traffic from `h1` to `h3` must traverse `SF1`, then `SF3`, then `SF2`;
- traffic from `h2` to `h4` must traverse `SF3`;
- return traffic must bypass the service chains and use ordinary IPv4 forwarding along the shortest path.

3 Packet Formats

The project uses three packet formats, depending on the portion of the topology where the packet is observed.

Hosts generate ordinary Ethernet/IPv4/TCP packets. When the classifier selects a service chain, it encapsulates the original Ethernet frame inside NSH and transports it through an MPLS tunnel. Therefore, packets exchanged between overlay elements use the following format:

```
Eth / MPLS / NSH / Eth / IPv4 / TCP
```

Packets exchanged between an SFF and a Service Function must not contain MPLS or NSH:

```
Eth / IPv4 / TCP
```

The same plain Ethernet/IPv4/TCP format must be used after the service chain has been completed and for all return traffic.

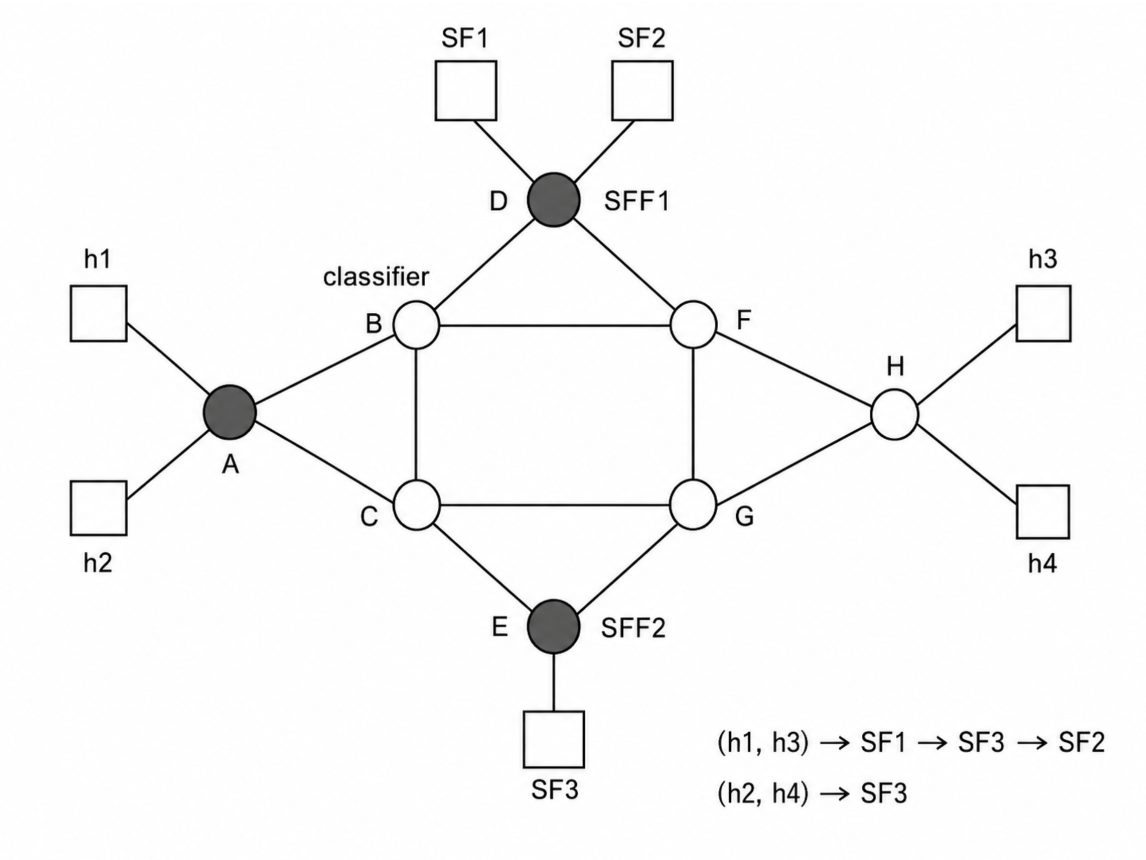

Figure 1: Reference SFC topology and packet formats.

Network segment	Packet format
Host to classifier	`Eth / IPv4 / TCP`
Classifier to SFF	`Eth / MPLS / NSH / Eth / IPv4 / TCP`
SFF to SFF	`Eth / MPLS / NSH / Eth / IPv4 / TCP`
SFF to SF	`Eth / IPv4 / TCP`
SF to SFF	`Eth / IPv4 / TCP`
End of chain to destination	`Eth / IPv4 / TCP`
Return traffic	`Eth / IPv4 / TCP`

Table 1: Packet formats used in the project.

4 SFC Classifier

The classifier receives ordinary IPv4 packets and determines whether they must enter a service chain. The classifier must inspect selected packet fields, such as source address, destination address, protocol, and TCP ports.

For packets matching an SFC policy, the classifier must:

1. insert an NSH header;
2. set the Service Path Identifier (SPI);
3. set the Service Index (SI);
4. preserve the original Ethernet/IPv4/TCP packet as the inner packet;
5. encapsulate the packet into the appropriate MPLS tunnel toward the next overlay element.

Packets that do not match any SFC policy must be forwarded using ordinary IPv4 forwarding or dropped, depending on the chosen configuration.

5 MPLS-Based Overlay Transport

MPLS is used to interconnect the overlay elements of the SFC architecture. IPv4 forwarding must not be used to transport packets between overlay elements once the packet has entered the SFC domain.

The MPLS label identifies the tunnel or transport path between overlay nodes. Students must implement the P4 logic required to push, pop, or swap MPLS labels, depending on their design.

The SFC overlay transport format is:

```
Eth / MPLS / NSH / Eth / IPv4 / TCP
```

6 Service Function Forwarder

Each SFF must process NSH-encapsulated packets and forward them according to the current service-chain state. The forwarding decision must be based on the pair:

```
SPI, SI
```

The SFF must support the following actions:

- forward the packet to a local Service Function;
- forward the packet to another SFF through an MPLS tunnel;
- forward the packet to the egress portion of the network after chain completion;
- drop the packet in case of invalid or unsupported state.

7 SFF Proxy Behaviour

Service Functions are not NSH-aware. Each Service Function is a simple P4 reflector that receives an ordinary Ethernet/IPv4/TCP packet and sends it back to the SFF.

When an SFF sends a packet to a Service Function, it must:

1. save or reconstruct the current service-chain context;
2. remove the MPLS header, if present;
3. remove the NSH header;
4. send the original Ethernet/IPv4/TCP packet to the Service Function.

When the packet returns from the Service Function, the SFF must:

1. recognize the packet as returning from a local Service Function;
2. restore the correct NSH context;
3. update the Service Index;
4. select the next service-chain hop;
5. encapsulate the packet into an MPLS tunnel if the next overlay element is remote.

The mechanism used to associate returning packets with their NSH context is part of the project and must be documented in the final report.

8 Service Functions

Each Service Function must be implemented as a P4 reflector. It must receive a packet from the attached SFF and send it back to the same SFF.

Optionally, a Service Function may perform a simple modification to make its traversal visible during testing. For example, it may modify the DSCP field, update a counter, or mark packets belonging to a specific flow.

Service Functions must not parse or process MPLS or NSH.

9 Return Traffic

Return traffic must not enter the service chains. For example, traffic from `h3` to `h1` and from `h4` to `h2` must be forwarded directly using IPv4 forwarding.

The return path must follow the shortest path in the underlying topology. Students may implement shortest-path forwarding using static IPv4 forwarding entries computed before running the experiment.

The return packet format is always:

```
Eth / IPv4 / TCP
```

10 Testing with iperf3

The implementation must be tested using `iperf3`. At minimum, students must run the following experiments:

- `h1` as client and `h3` as server, verifying that the forward traffic traverses `SF1`, `SF3`, and `SF2`;
- `h2` as client and `h4` as server, verifying that the forward traffic traverses `SF3`;
- reverse-direction traffic, verifying that it bypasses the service chains and follows the shortest IPv4 path;
- packet captures on SFF–SF links, verifying that Service Functions receive only `Eth / IPv4 / TCP`;
- packet captures on overlay links, verifying that overlay packets use `Eth / MPLS / NSH / Eth / IPv4 / TCP`.

10.1 MTU and MSS

Because the SFC overlay adds MPLS, NSH, and an additional inner Ethernet header, the effective MTU available to the original IPv4/TCP packet is reduced. Students must avoid fragmentation during `iperf3` tests.

Assuming a 1500-byte link MTU, one MPLS label, an 8-byte simplified NSH header, and a preserved inner Ethernet header, the maximum inner IPv4 packet size is:

```
1500 - 4 - 8 - 14 = 1474 bytes
```

Assuming IPv4 without options and TCP without options, the corresponding TCP MSS is:

```
1474 - 20 - 20 = 1434 bytes
```

Therefore, a safe `iperf3` command is:

```
iperf3 -c <server-ip> -M 1434
```

If a larger NSH header is implemented, the MSS must be reduced accordingly. For example, with a 24-byte NSH header, the suggested MSS is:

```
1500 - 4 - 24 - 14 - 20 - 20 = 1418 bytes
```

Students must state the assumed header sizes and the MSS used in their experiments.